

Major Project Report

On

MCP BASED PERSONAL PRODUCTIVITY AGENT

Submitted to Kommuri Pratap Reddy Institute of Technology for the partial fulfillment
of the Requirement for the Award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering (AI&ML)

by

KRISHNA KEERTHANA POTHARAJU (23RA1A6641)

SOMA GANESH (23RA1A6627)

NIKHIL KOKKULA (23RA1A6662)

MALISHKA REDDY GONE (23RA1A6647)

Under the guidance of

**HoD, Dr. PULIME SATYANARAYANA,
M.Tech., Ph.D**

Associate Professor of CSE(AI&ML)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(An UGC Autonomous Institution, Affiliated to JNTUH, Hyderabad)

Ghanpur (V), Ghatkesar (M), Medchal (D)-500088

2026-2027

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(An UGC Autonomous Institution, Affiliated to JNTUH, Hyderabad)

Ghanpur (V), Ghatkesar (M), Medchal (D)-500088

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)



CERTIFICATE

This is to certify that the dissertation entitled **“MCP BASED PERSONAL PRODUCTIVITY AGENT”** being submitted by **KRISHNA KEERTHANA POTHARAJU (23RA1A6641), SOMA GANESH (23RA1A6627), NIKHIL KOKKULA (23RA1A6662), MALISHKA REDDY GONE(23RA1A6647)** in partial fulfillment for the award of Bachelor of Technology in Computer Science and Engineering (AI&ML) to the Kommuri Pratap Reddy Institute of Technology is a record of confined work carried out by them under my guidance and supervision.

Guide's signature

Dr. PULIME SATYANARAYANA

M. Tech., Ph.D.

Associate Professor

Dept. of CSE (AI&ML)

Signature of the Head of the Dept.

Dr. PULIME SATYANARAYANA

M. Tech., Ph.D.

Professor

Dept. of CSE (AI&ML)

External Examiner

Place:Ghanpur

Date:

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(An UGC Autonomous Institution, Affiliated to JNTUH, Hyderabad)

Ghanpur (V), Ghatkesar (M), Medchal (D)-500088

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)



DECLARATION

We, **KRISHNA KEERTHANA POTHARAJU (23RA1A6641), SOMA GANESH (23RA1A6627), NIKHIL KOKKULA (23RA1A6662), MALISHKA REDDY GONE(23RA1A6647)** here by declare that the project report titled **“MCP-BASED PERSONAL PRODUCTIVITY AGENT”** under the guidance of **Dr.Pulime Satyanarayana**, Associate Professor of Computer Science and Engineering(AI&ML), Kommuri Pratap Reddy Institute of Technology, Ghanpur, is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI&ML).

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this thesis have not been submitted to any other University and Institute for the award of any Degree or Diploma.

SUBMITTED BY

KRISHNA KEERTHANA POTHARAJU(23RA1A6641)

SOMA GANESH (23RA1A6627)

NIKHIL KOKKULA (23RA1A6662)

MALISHKA REDDY GONE (23RA1A6647)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without acknowledging the individuals who made it possible.

We are extremely indebted to our internal guide, **Dr. PULIME SATYANARAYANA, M.Tech., Ph.D** Associate Professor, for his valuable help. We would like to express our profound gratitude to her for providing us the opportunity to work under her supervision and guidance. His guidance helped us in the successful completion of my project.

We would like to express our heartfelt gratitude to our **HoD, Dr. PULIME SATYANARAYANA, M.Tech., Ph.D** Associate Professor, for providing the guidance and necessary infrastructure to complete this project.

We would also like to express our deep-felt gratitude to our **DIRECTOR, Dr. B. SUDHEER PREM KUMAR, M.Tech., Ph.D** of Kommuri Pratap Reddy Institute of Technology, for his kind cooperation and valuable suggestions.

We would also like to express our deep-felt gratitude to **Dr. SRINATH KASHYAP, M.Tech., Ph.D.,** **PRINCIPAL** of Kommuri Pratap Reddy Institute of Technology, for his kind cooperation and valuable suggestions.

This Acknowledgement will be incomplete without mentioning our sincere gratefulness to our honorable **CHAIRMAN Shri. KOMMURI PRATAP REDDY,** and **VICE-CHAIRMAN Shri KOMMURI PRASHANTH REDDY** of Kommuri Pratap Reddy Institute of Technology for providing the necessary infrastructure to complete this project.

We also thank the staff of the CSE(AI&ML) department who helped us morally for the successful completion of this project.

SUBMITTED BY

KRISHNAKEERTHANAPOTHARAJU	(23RA1A6641)
SOMA GANESH	(23RA1A6627)
NIKHIL KOKKULA	(23RA1A6662)
MALISHKA REDDY GONE	(23RA1A6647)



Kommuri Pratap Reddy Institute of Technology
Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning)

Vision of the Institute

To emerge as a premier institute for high quality professional graduates who contribute to economic and social developments of the Nation.

Mission of the Institute

Mission	Statement
IM1	To have holistic approach in curriculum and pedagogy through industry interface to meet the needs of Global Competency.
IM2	To develop students with knowledge, attitude, employability skills, entrepreneurship, research potential and professionally Ethical citizens.
IM3	To contribute to advancement of Engineering & Technology that would help to satisfy the societal needs.
IM4	To preserve, promote cultural heritage, humanistic values and Spiritual values thus helping in peace and harmony in the society.



Kommuri Pratap Reddy Institute of Technology
Department of Computer Science and Engineering (Artificial
Intelligence & Machine Learning)

Vision of the Department

To be a premier center of excellence with research in Artificial Intelligence and Machine Learning research through unique interdisciplinary partnerships and positive contribution to the students, community, organizations and society as a whole.

Mission of the Department

Mission	Statement
DM1	To impart knowledge in cutting edge Artificial Intelligence technologies in par with industrial standards.
DM2	To collaborate with industry to uplift innovative research and development in Artificial Intelligence and Machine Learning and its allied fields to serve the needs of society.
DM3	To produce successful Computer Science and Engineering graduates with a specialization in AIML with personal and professional responsibilities and commitment to lifelong learning.



Program Outcomes

PO1	Engineering Knowledge: Apply the knowledge of mathematics, science, Engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental knowledge of, and need for sustainable development.

PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and team network: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, being able to comprehend and write effective reports and design documentation, make Effective presentations, and give and receive clear instructions.
PO11	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environment.
PO12	Life-Long learning: Recognize the need for, and have the preparation and able to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes

PSO1	Demonstrate the knowledge of human cognition, Artificial Intelligence, Machine Learning and data engineering for designing intelligent systems.
PSO2	Apply computational knowledge and project development skills to provide innovative solutions.
PSO3	Use tools and techniques to solve problems in AI & ML.

ABSTRACT

Managing tasks, schedules, notes, emails, and personal information across multiple applications can be time-consuming and fragmented. **MCP-Based Personal Productivity Agent** addresses this problem by creating a unified AI assistant that can intelligently access and interact with different productivity tools through **Model Context Protocol (MCP)**.

The system uses an **AI agent built with LangGraph** to understand user requests, discover the appropriate MCP tools, and perform actions such as checking schedules, managing tasks, searching notes, or retrieving relevant information. Instead of hard-coding individual integrations, MCP provides a standardized interface that makes the agent's tool usage more flexible, modular, and scalable.

The platform can be developed using **Python, FastMCP, LangGraph, Streamlit**, and synthetic or personal productivity data. Custom MCP servers can provide tools for **calendar management, task tracking, and note searching**, while the conversational interface allows users to interact naturally with the agent.

By combining **Agentic AI, standardized tool integration, and natural-language interaction**, the system demonstrates how MCP can simplify AI-tool communication and enable intelligent assistants that are more **modular, extensible, and adaptable** to different productivity workflows.

Keywords: Model Context Protocol (MCP), Agentic AI, Dynamic Tool Discovery, AI Tool Orchestration,

LangGraph, Natural Language Interaction, Modular AI Architecture, Productivity Automation

INTRODUCTION

Personal productivity has become increasingly dependent on digital tools for managing tasks, schedules, reminders, notes, communication, and information. Although numerous applications are available for these activities, users often need to move between multiple platforms to complete simple workflows. This fragmentation can increase cognitive load, consume time, and make it difficult to maintain a consistent view of pending activities. Artificial intelligence provides an opportunity to simplify these interactions by allowing users to communicate with productivity systems using natural language.

A personal productivity agent can act as an intelligent intermediary between the user and different digital services. Instead of manually navigating individual applications, a user can provide a request such as creating a task, checking a schedule, setting a reminder, or retrieving information. The agent can understand the intent behind the request and select the appropriate tool to accomplish it. For such an approach to work effectively, the AI system requires a reliable mechanism for discovering and interacting with external tools.

The Model Context Protocol (MCP) provides a standardized approach for connecting AI applications with external tools, resources, and services. In an MCP-based architecture, an AI agent can communicate with MCP servers that expose specific capabilities through well-defined interfaces. This separation between the reasoning component and individual tools makes the system easier to extend and maintain. New productivity capabilities can be added by connecting additional MCP-compatible services without redesigning the entire agent.

The proposed MCP-Based Personal Productivity Agent focuses on combining conversational AI with tool-based productivity automation. The system is intended to understand user requests, determine the required action, interact with appropriate MCP tools, and provide a clear response to the user. By bringing multiple productivity operations into one interface, the project aims to provide a more efficient, convenient, and user-friendly approach to personal task management.

LITERATURE SURVEY

Toolformer (2023): LLMs can dynamically use external APIs or tools instead of relying only on internal knowledge. The study shows that a language model can learn when to call a tool, select the required API, provide arguments, and use the returned result in its response. This demonstrates the value of tool-using LLMs for tasks requiring calculation, search, and external information.

Limitation: it does not provide a standardized and modular protocol for connecting many independently developed tools.

Agent Interoperability Survey (2025): This survey studies MCP and other protocols for interoperable agents and compares their approaches to communication, tool or agent discovery, interaction, and security. It highlights the need for standardized interfaces because ad-hoc integrations become difficult to maintain as the number of tools increases. MCP is relevant to productivity agents because it provides a structured way for an AI agent to interact with external tools. **Limitation:** the study is mainly a protocol-level analysis rather than a complete personal productivity implementation.

Tool Preferences in Agentic LLMs (2025): This study examines how agentic LLMs select tools and shows that tool selection can be unreliable and strongly dependent on the descriptions provided for available tools. This is important when an agent has several tools with similar capabilities, because unclear descriptions may lead to incorrect selection. **Limitation:** reliable tool selection cannot be assumed only from the availability of a protocol; clear tool descriptions and suitable agent orchestration are also required.

Literature Gap: The reviewed works establish tool-using LLMs, interoperable agent protocols, and the challenges of tool selection. However, they do not directly provide a unified personal productivity system combining conversational interaction with MCP-based task, calendar, reminder, and note-management tools. The proposed project addresses this gap through a modular MCP-based productivity agent.

PROBLEM STATEMENT

Modern users rely on multiple digital applications to manage their personal, academic, and professional activities. Tasks, calendars, reminders, notes, and other information are frequently maintained in separate systems, requiring users to repeatedly switch between applications and manually perform routine operations. This fragmented workflow can lead to missed tasks, duplicated effort, inefficient time management, and increased cognitive load.

Existing productivity applications generally provide useful individual features, but they do not always offer a unified conversational mechanism for coordinating multiple tools. Conventional automation systems may also require predefined workflows and technical configuration, which can limit flexibility for users who want to express requests naturally. There is therefore a need for an intelligent productivity solution that can understand natural-language instructions and interact with multiple productivity services through a standardized tool interface.

The problem addressed by this project is the development of an MCP-Based Personal Productivity Agent capable of receiving natural-language requests, understanding user intent, selecting suitable external tools, and coordinating productivity-related actions through MCP. The system should provide a unified and user-friendly interaction layer while maintaining a modular architecture that allows additional tools and services to be integrated in the future.

EXISTING SYSTEM

Existing personal productivity systems generally depend on separate applications for managing tasks, calendars, notes, reminders, emails, and other daily activities. Users must manually open each application and perform the required operation. This fragmented approach makes it difficult to manage multiple activities from a single interface.

Traditional productivity applications are usually designed around predefined features and workflows. They may provide task management or calendar functionality, but they often have limited ability to understand natural-language instructions and execute actions across different services. When users need to connect multiple applications, separate APIs and custom integration logic are generally required.

Existing AI assistants provide conversational interaction and can understand user queries, but their ability to perform actions is often restricted by the tools and integrations available to them. Adding new services can require additional development and configuration. As a result, existing systems may not provide a flexible, standardized, and easily extensible architecture for connecting an AI agent with multiple productivity tools.

PROPOSED SYSTEM

The proposed system, “MCP-Based Personal Productivity Agent,” is designed to provide a unified and intelligent interface for managing personal productivity activities. The system uses an AI-based agent that understands natural-language instructions and determines the appropriate action to perform.

The Model Context Protocol (MCP) is used as the communication layer between the AI agent and external tools or services. Through MCP, productivity tools can expose their capabilities in a standardized manner, allowing the agent to discover and invoke the required tools without requiring a completely different integration mechanism for each service.

For example, a user can provide a request such as “Create a task to complete my project report tomorrow.” The AI agent interprets the request, identifies the appropriate productivity tool, and sends the required operation through the MCP interface. Similarly, the system can be extended to support calendar events, reminders, notes, information retrieval, and other productivity-related operations.

The proposed system aims to reduce repetitive manual work, minimize application switching, improve productivity, and provide a natural conversational experience. Its modular architecture also makes it easier to add new tools and services in the future.

HARDWARE REQUIREMENTS

The proposed MCP-Based Personal Productivity Agent does not require specialized hardware and can be developed and executed on a standard computer system. The recommended hardware requirements are:

- Processor: Intel Core i5 or equivalent AMD processor and above
- RAM: Minimum 8 GB; 16 GB recommended for development and testing
- Storage: Minimum 256 GB SSD
- Display: 1366 × 768 resolution or higher
- Network: Stable broadband or Wi-Fi connection
- Input Devices: Standard keyboard and mouse
- Optional: Microphone for future voice-based interaction

The system may require a stable internet connection when using cloud-based Large Language Models (LLMs), external APIs, or remote MCP services.

SOFTWARE REQUIREMENTS

The software requirements for developing and operating the MCP-Based Personal Productivity Agent include:

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python
- Development Environment: Visual Studio Code or another suitable IDE
- AI/LLM Integration: An API or locally hosted Large Language Model
- MCP: Model Context Protocol-compatible client and MCP servers
- Backend Framework: FastAPI, Flask, or an equivalent Python framework
- Database: SQLite, PostgreSQL, or another suitable database depending on implementation
- Version Control: Git and GitHub
- API Testing: Postman or equivalent tool
- Web Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or equivalent
- Internet Connection: Required for cloud-based AI models and external productivity services

The exact software stack can be selected according to the implementation requirements and the productivity tools integrated into the system.

1. SYSTEM ARCHITECTURE

The MCP-Based Personal Productivity Agent follows a layered architecture in which each layer has a clearly defined responsibility. The user provides a natural-language productivity request, and the LangGraph Agent controls the reasoning workflow. The LLMClient supports intent understanding and response generation, while the MCP Client provides standardized tool discovery and invocation. The Unified MCP Server acts as the single gateway for 17 productivity tools covering Tasks, Calendar, and Notes. These tools remain thin and delegate business logic to the corresponding service layer. The service layer performs validation, business rules, conflict handling, search, and persistence through SQLAlchemy and SQLite. This separation keeps reasoning, protocol communication, domain logic, and storage independently testable and extensible.

Layer	Responsibility
Presentation / User Interaction	Accepts natural-language productivity requests and displays the final response.
Agent / Reasoning Layer	Understands intent, selects tools, prepares arguments, validates execution, and controls the workflow using LangGraph.
LLM Integration Layer	Provides the LLMClient abstraction for structured request interpretation and final response generation.
MCP Client	Connects through stdio, discovers capabilities dynamically, invokes tools, and manages the MCP session lifecycle.
Unified MCP Server	Provides one standardized interface exposing exactly 17 productivity tools.
Domain MCP Tools	Translate MCP requests into Task, Calendar, and Notes service operations without duplicating business logic.
Service / Validation Layer	Owns input validation, business rules, date handling, conflict detection, search, persistence, and controlled errors.
Database Layer	Uses SQLAlchemy ORM with SQLite to store and retrieve tasks, calendar events, and notes.

Table 1: Layered architecture and responsibilities

Architecture interpretation: The key design principle is separation of concerns. The agent determines what the user wants, the MCP layer provides standardized capability access, the service layer determines how the business operation is performed, and the database layer persists the result. The Unified MCP Server does not directly implement domain business logic or database queries.

2. MODULE ARCHITECTURE

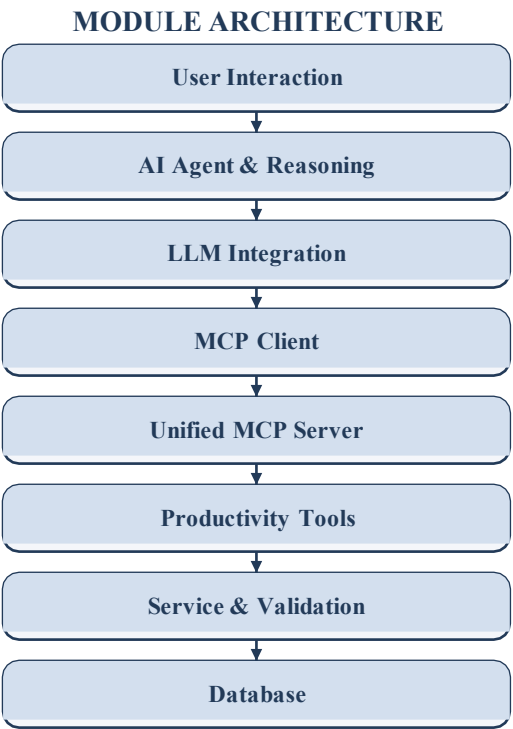


Figure 1: Major module flow of the MCP-based productivity system

The module structure follows the implemented layered flow of the system. Natural-language input enters through the user interaction layer and is processed by the AI Agent and LLM integration. The MCP Client provides dynamic capability discovery and invocation through the Unified MCP Server. The server exposes the Task, Calendar, and Notes productivity tools, which delegate domain operations to the service and validation layer. Finally, SQLAlchemy and SQLite provide persistent storage. This separation keeps reasoning, protocol communication, business logic, and data storage independent and reusable.

2.1 MODULE DESCRIPTION

The following module descriptions represent the major functional parts of the implemented system.

1) User Interaction

Accepts natural-language productivity requests and displays the final response. The user can request task, calendar, or notes operations without knowing the underlying MCP tool names.

2) AI Agent & Reasoning

Understands user intent and determines the required action using LangGraph. It coordinates tool discovery, selection, argument preparation, validation, invocation, result handling, and response generation.

3) MCP Client

Connects with MCP servers, initializes sessions, discovers available tools dynamically, invokes the selected tool with structured arguments, and manages connection lifecycle and failures.

4) Unified MCP Server

Provides a single standardized interface for all productivity tools. It exposes exactly 17 tools: 6 Task tools, 5 Calendar tools, and 6 Notes tools.

5) Productivity Tools

Handles Tasks, Calendar, and Notes operations according to the user's request. The tools provide CRUD capabilities, task completion, and note search while delegating business logic to services.

6) Service & Validation

Performs input validation, business rules, date/time handling, conflict detection, search, persistence, and controlled error handling. It remains the source of truth for domain behaviour.

7) Database

Stores and retrieves tasks, calendar events, and notes using SQLAlchemy ORM with SQLite persistent storage.

3. UML DIAGRAMS

UML (Unified Modeling Language) diagrams describe the structure and behaviour of the system visually. Six diagrams are used here: the use case diagram for functionality, the class diagram for static structure, the sequence diagram for message order, the activity diagram for workflow, the component diagram for architecture, and the ER diagram for the persistent data model.

3.1 Use Case Diagram

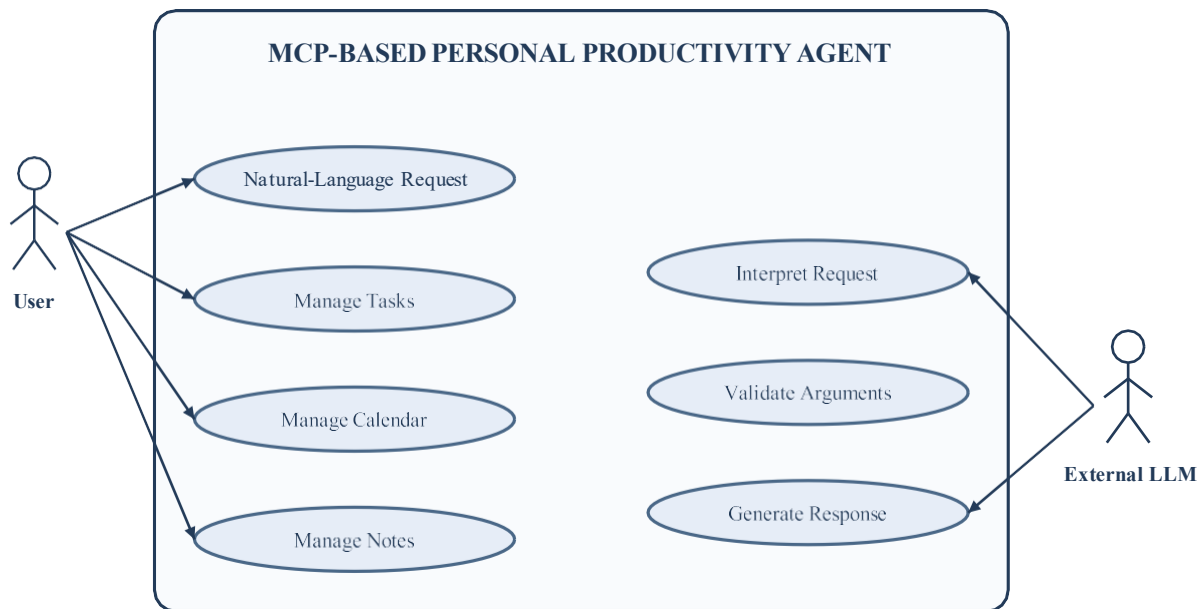


Figure 2: Use case diagram of the MCP-based personal productivity agent

The use case diagram shows the functional requirements from the point of view of the user and the external language-model dependency. The user initiates natural-language productivity requests and can work with Tasks, Calendar, and Notes. The agent-side use cases cover request interpretation, validation, and response generation, while the external LLM provider supports language understanding and response generation. Internal components such as the MCP Client, services, and database are implementation components rather than UML actors. This keeps the system boundary clear and focuses the diagram on user-visible functionality.

3.2 Class Diagram

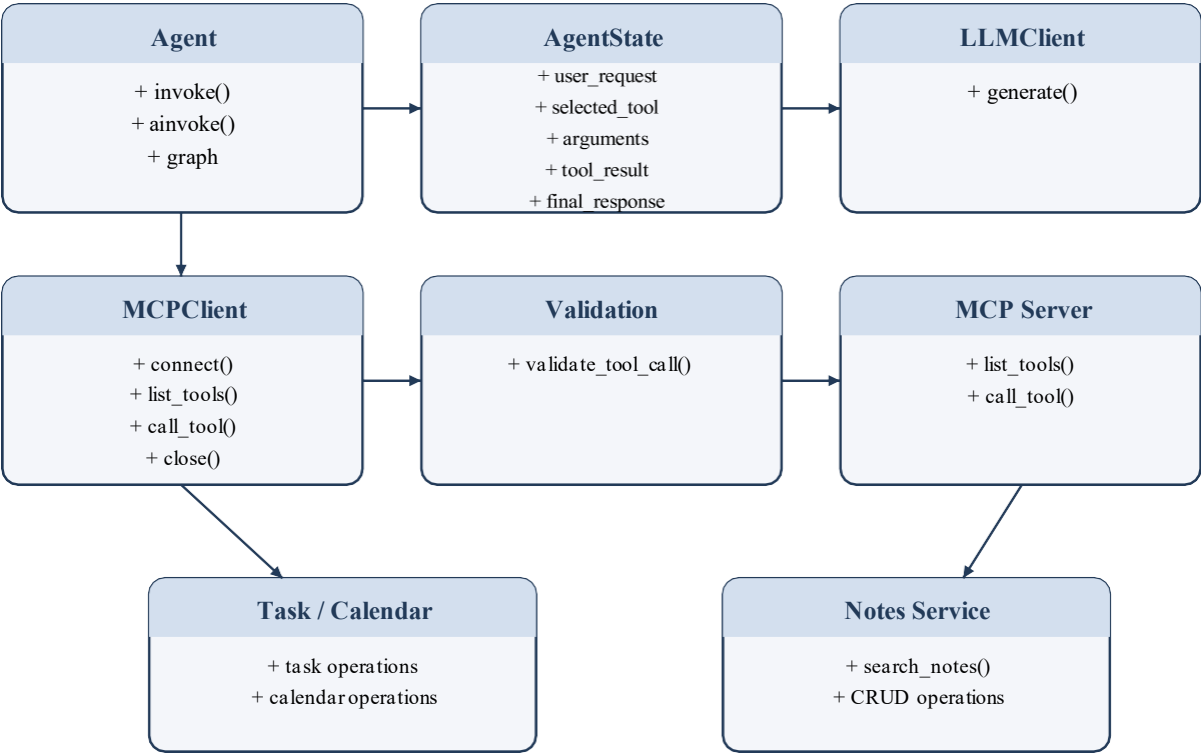


Figure 3: Class diagram of the agent, MCP and service structure.

The class diagram presents the static structure of the core software abstractions. Agent controls the LangGraph workflow, while AgentState carries the information needed between workflow steps. LLMClient isolates language-model communication, MCPClient handles connection and tool invocation, and Validation provides the pre-execution validation boundary. The MCP Server exposes capabilities and delegates domain work to Task, Calendar, and Notes services. The database is accessed through the service layer using SQLAlchemy and SQLite, keeping persistence separate from agent logic.

3.3 Sequence Diagram

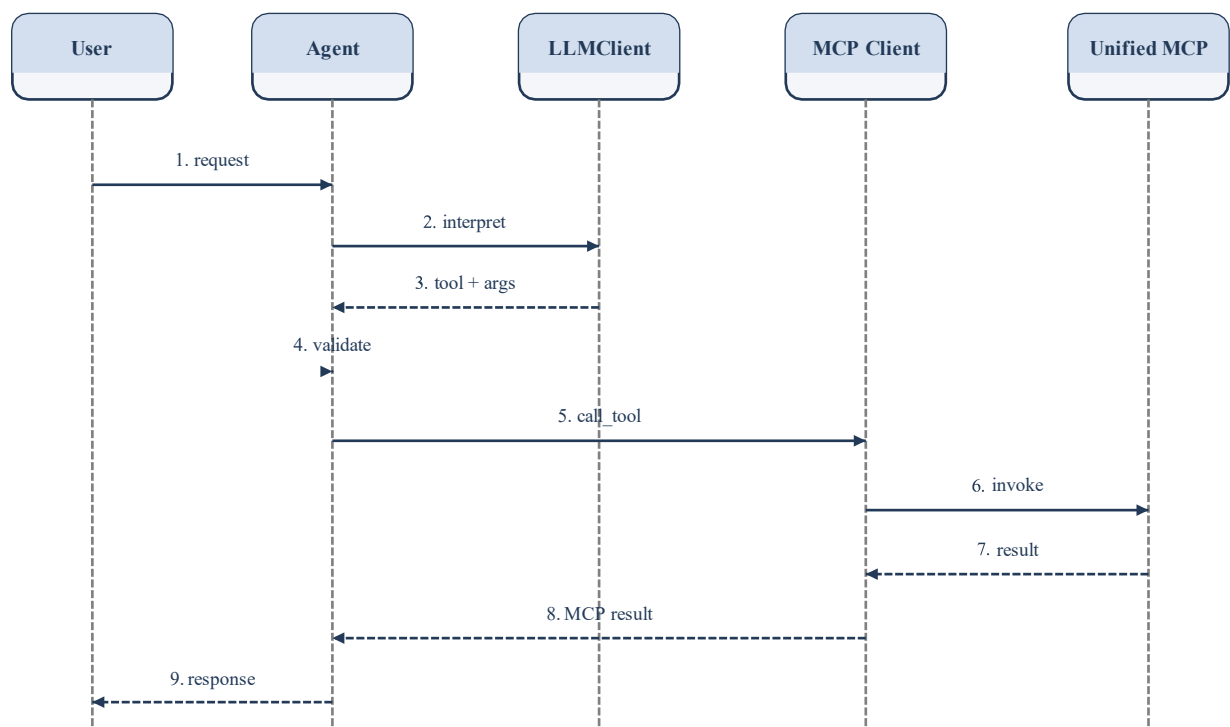


Figure 4: Sequence diagram of one natural-language productivity request

The sequence diagram shows the order of messages for a single productivity request. The user sends a natural-language request to the agent, which uses the LLMClient to interpret the request and determine the tool and arguments. The agent validates those arguments before the MCP Client invokes the selected tool on the Unified MCP Server. The server-side operation returns a structured result through the MCP connection. The agent then converts that result into the final response shown to the user.

3.4 Activity Diagram

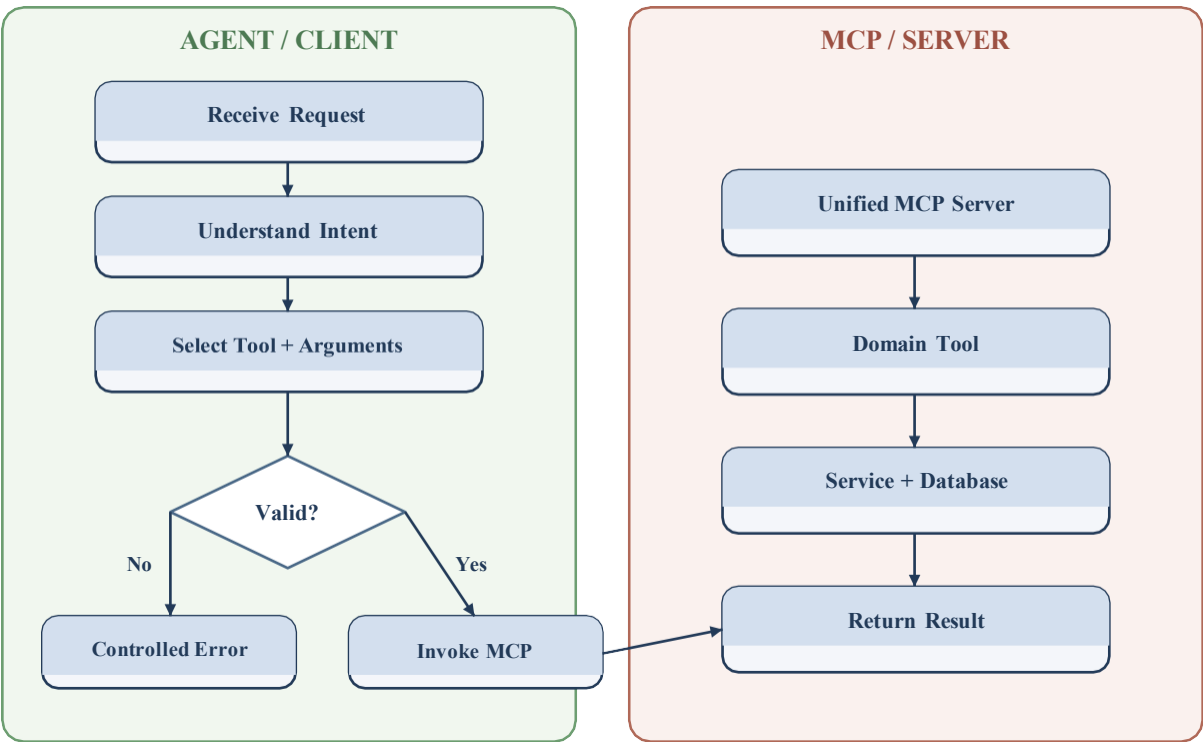


Figure 5: Activity diagram of the end-to-end natural-language workflow

The activity diagram represents the complete control flow from request to response. The agent receives the request, understands the intent, selects the appropriate tool, and prepares its arguments. A decision node checks whether the arguments are valid. Invalid input produces a controlled error, while valid input continues to MCP invocation. The Unified MCP Server routes the call to the domain tool and service, and the resulting data returns to the agent for final response generation.

3.5 Component Diagram

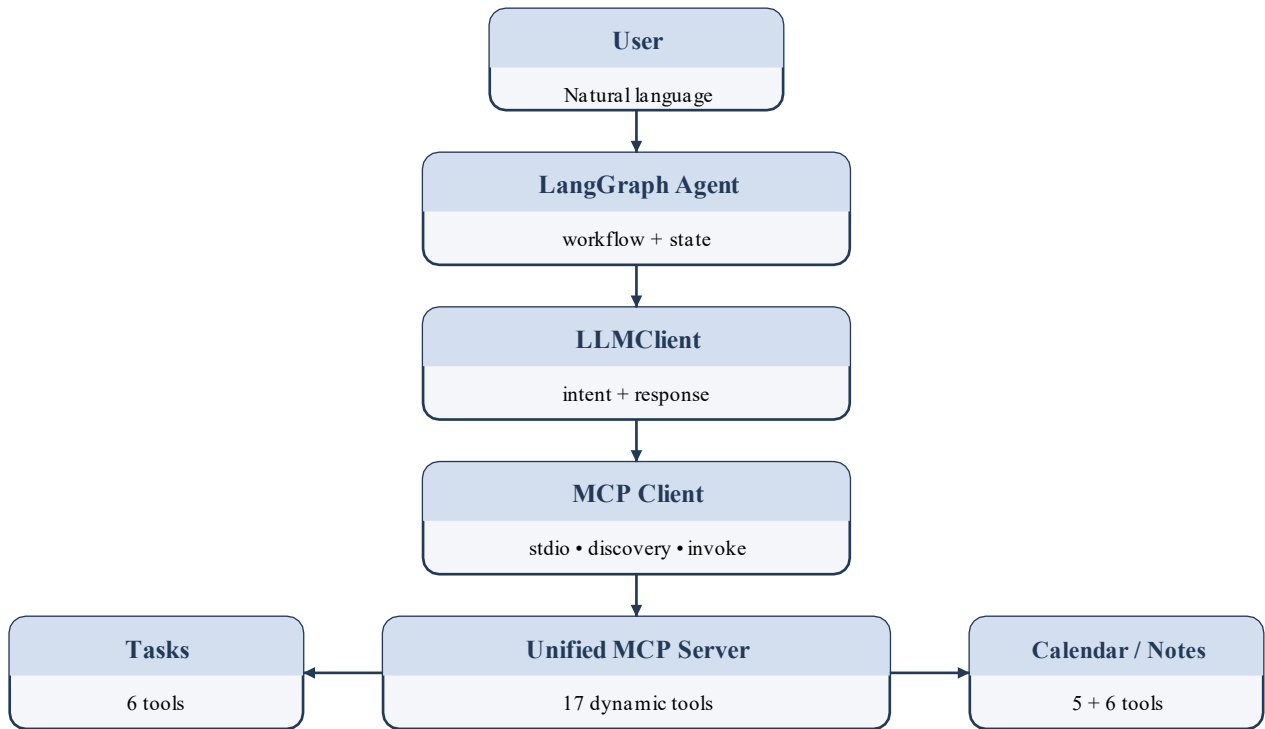


Figure 6: Component diagram of the layered MCP agent architecture

The component diagram presents the runtime architecture and dependency direction between the major components. The LangGraph Agent and LLMClient handle reasoning and language interaction, while the MCP Client handles standardized capability access. The Unified MCP Server exposes 17 tools and routes them to the Task, Calendar, and Notes domains. The service layer contains business rules and validation, while SQLAlchemy and SQLite provide persistence. This separation prevents the agent from directly accessing the database and keeps the MCP server thin.

3.6 ER Diagram

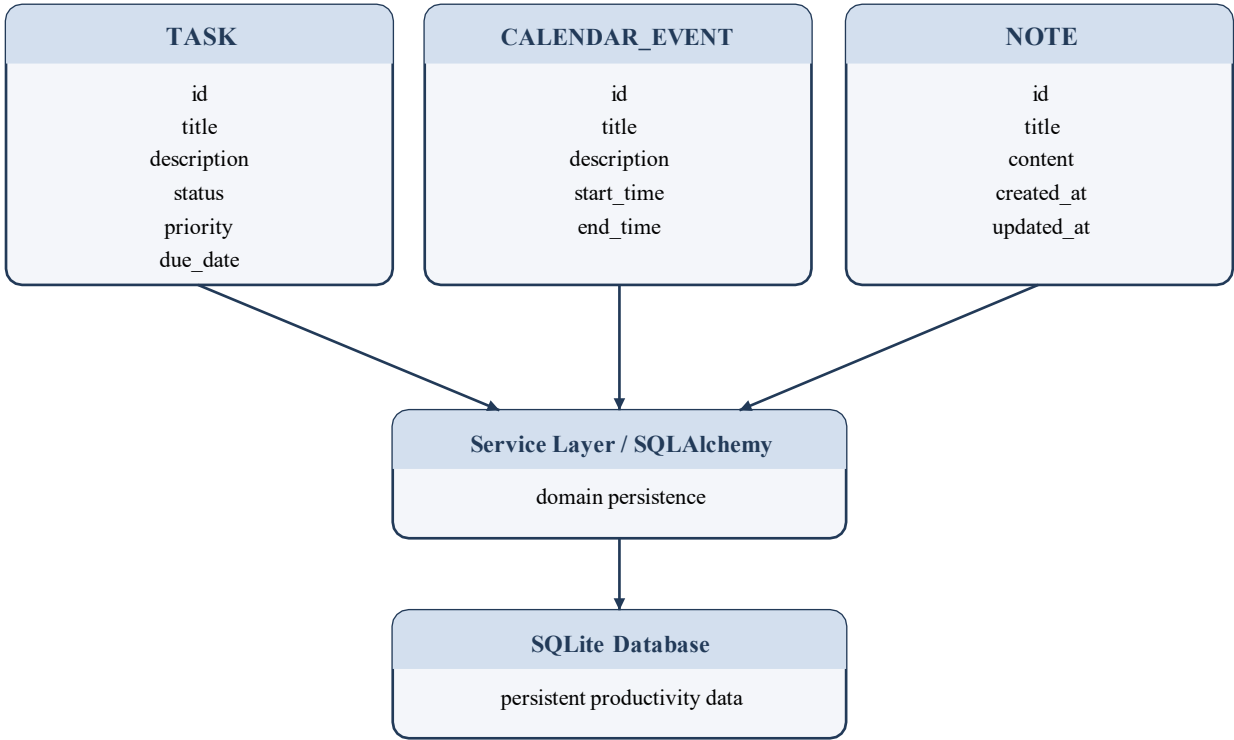


Figure 7: ER-style data model of the productivity domains

The ER-style diagram represents the three core persistent productivity entities. Task stores information such as title, status, priority, and due date. Calendar Event stores scheduling information through start and end times, while Note stores textual content and timestamps. Each domain is handled by its corresponding service and exposed through MCP tools. SQLAlchemy maps these application entities to SQLite persistence, keeping the data layer separate from the agent and protocol layers.

4. DESIGN SUMMARY

The six UML views collectively document the project from complementary perspectives. The use case diagram explains what the user can accomplish, the class diagram shows the main software abstractions, and the sequence diagram explains message order during execution. The activity diagram presents the decision and validation workflow, the component diagram shows the runtime architecture, and the ER diagram presents the persistent Task, Calendar, and Notes data model. Together, these views describe how the LangGraph agent, LLMClient, MCP Client, Unified MCP Server, 17 productivity tools, service layer, and SQLite persistence cooperate as one modular productivity system.